University of Cape Town

ECO5040W — Financial Software Engineering

Group Rationale

XRPL-Based FX Remittance Platform using RLUSD

Annita Ngoma Karabo Tigedi Kerry-Lynn Whyte

Liltha Mzamo Nikola Milosavljevic

Group 3

7 September 2026

This note is for the group and for anyone marking the repo. It explains, in ordinary language, why remittance giants look the way they do, what this platform is trying to do instead, how the XRP Ledger pieces actually work, what was wrong with the first GitHub sketch, why we patched settlement before quotes, and how to run a demo on Windows. The system of record is this fork (Karabo22Tigedi / FSE-Project), not the sketch. Reports describe the patched API. One hundred and twenty-nine tests pass.

1 How Western Union and MoneyGram work

If you have ever sent money home from a South African mall, you already know the shape. You queue at a Western Union or MoneyGram desk, show an ID, fill in the recipient's name and city, and pay rand. The clerk quotes a fee and a payout amount in the other currency. You get a reference number (MTCN at Western Union; a similar reference at MoneyGram). You phone or WhatsApp that number to the recipient. They go to an agent on their side, show ID, and collect cash.

Behind the counter there is a licensed money transmitter, an agent network, a compliance file, and a chain of banks that move value between countries. The customer does not see correspondent accounts or nostro/vostro balances. They see a fee, a rate, a wait, and cash. Both brands also offer digital send from a bank account or card, but a large share of corridors that matter for southern Africa still end in cash at an agent (FinMark Trust, 2024; International Monetary Fund, 2025).

The product is not mysterious. It is *cash in, a message that value has been paid, cash out*, with the operator sitting in the middle taking a fee for the network, the compliance, and the guarantee that the agent will pay. Our prototype copies that shape: rand in (simulated), a settlement instruction, digital value in a wallet, then a simulated cash-out. The middle hop is an RLUSD (or UCTUSD) Payment on the XRP Ledger Testnet instead of a bank wire (University of Cape Town, 2026a).

Western Union's MTCN and MoneyGram's reference are how the two sides of that cash sandwich find each other without sharing a bank. Our `tracking_ref` (`MG` plus ten digits) is the same idea in

miniature: the sender can WhatsApp it, the recipient or an admin can look it up, and it is not a 36-character UUID. The important difference is what happens between the two cash doors. In the traditional model that interval is the operator's internal books plus correspondent banks. In ours it is a Testnet Payment whose hash is stored on the remittance row.

2 Where they fail the customer

They do not fail at existing. They fail at price, speed, and clarity for small sends.

The World Bank's Remittance Prices Worldwide series is the public benchmark for "how much does it cost to send a few hundred dollars home?" (World Bank, 2025). South Africa is repeatedly a high-cost source market. Rateweb's write-up of the 2025 Q1 figures puts Western Union and MoneyGram flat fees around R214–R217 on a surveyed send of about R1,370 (Rateweb, 2026). That is not a rounding error. It is more than a tenth of the money before the operator's FX spread.

The IMF corridor diagnostic for South Africa–Zimbabwe and FinMark's SADC remittance work describe the rest of the pain: cash collection, agent hours, delays, and prices that are hard to see until you are already at the till (FinMark Trust, 2024; International Monetary Fund, 2025). If the recipient does not have a bank account, cash is still the feature. The cost of that feature is paid by people sending grocery money.

A stablecoin does not delete those problems. Someone still has to take the rand and someone still has to hand over dollars or local cash. What a ledger *can* do is settle the *middle* quickly, with a public transaction hash, and with a fee model you can write down in four lines. That is the slice we built. We are honest about the rest: our cash-in and cash-out are simulated, and a token hop is not a SARB licence (University of Cape Town, 2026a).

3 How this platform is supposed to help

We are not replacing agent networks in production. We are showing a cleaner middle:

- A quote that states the fixed fee, the percentage fee, the FX margin on the rate, the RLUSD that will be paid, and an estimated cash-out. Admins can change those parameters without a code push.
- A 15-minute quote that the sender can cancel. Abandoned or expired quotes do not eat the R3,000 / R25,000 verified limits.
- A tracking reference that looks like a MoneyGram-style code (`MG` plus ten digits), not a UUID.
- After simulated cash-in is confirmed, a background worker pays RLUSD on Testnet from a treasury wallet into a custodial wallet that belongs to that recipient.
- The recipient sees spendable balance versus a computed on-chain figure, plus the Payment hash.

On a R1,370 send at the seeded defaults and the 18.50 fallback rate, the transaction fee is R38.70, not R214. That is an illustration with simulated rails, not a promise that a licensed product would charge R38.70. The point is that the formula is public and the sketch's "quotes forever consume limit" behaviour is gone.

We also give the recipient a custodial Testnet wallet they can log into, rather than only a payout PIN. They can hold the IOU, inspect the Payment hash on an explorer, or ask for a simulated USD or ZAR cash-out. That is closer to a fintech wallet than to a collect-at-the-pharmacy model. It still needs a real cash-out partner if it were ever more than coursework.

4 What we are not claiming

A ledger hop does not make Western Union obsolete, and it does not make us a bank. We do not operate agents, we do not take rand, we do not pay out dollars, and we do not have a licence. We do not run an AML rules engine. SQLite is not a production ledger. Tests mock XRPL. Redis has to be running or the worker has nothing to read. Those limits are documented in the specification so a marker is not sold a story the code cannot keep (University of Cape Town, 2026a).

5 XRPL issued tokens, in course language

XRP is the only asset every XRPL account can hold without asking. Everything else—RLUSD, a university IOU, a game token—is an *issued currency*. The course resources call these trust-line tokens, or informally IOUs: a promise from an issuing account that it owes the holder that amount of the underlying asset (University of Cape Town, 2026b; XRPL Foundation, 2026a).

Because an IOU is a promise, you cannot be forced to accept one. You opt in with a *trust line*: a TrustSet transaction that says "I am willing to hold up to X of this issuer's token" (XRPL Foundation, 2026b). Creating the line reserves a little XRP in the account (locked, not spent). Rippling is a related issuer setting that lets the same IOU move between holders; we do not implement rippling ourselves, but it is why an issuer usually turns Default Ripple on (University of Cape Town, 2026b).

Official RLUSD liquidity on Testnet is tight, so the course told us to keep issuer address and currency code in `.env`. We default to the UCTUSD fallback IOU. Switching to real RLUSD is a config change, not a rewrite (University of Cape Town, 2026b).

Our custody model is hybrid. One platform treasury wallet holds the scarce IOU. Each recipient still gets a real Testnet account, funded from a faucet, with a TrustSet to that issuer, so the Payment the worker submits is a real ledger transaction with a hash the recipient can look up. We persist the new address and encrypted secret *before* we submit TrustSet. If TrustSet fails, we retry with the same wallet; we do not generate a second one and strand the first.

None of this is Mainnet, and none of it is the recipient holding keys in Xaman. The platform is the custodian: it encrypts the seed with a Fernet key that is not in the database, and it signs Payments on the recipient's behalf. That is why the specification spends a page on custody and licensing. It is also why `setup_platform_wallet` prints the classic address and never the secret.

6 What the sketch got wrong

Kerry's repository was a useful sketch: FastAPI, bcrypt, Redis Streams, Testnet Payments, simulated cash legs. It also had correctness bugs we would not want to demo. This section is about *that* code, not about the current fork.

Abandoned quotes ate limits. A quote counted toward daily and monthly ZAR the moment it was created. There was no expiry and no cancel. A sender who asked for a price and walked away still burned their R3,000 day. That is not how a teller quote works, and it made the limit table lie.

Crash double-pay. If the worker submitted an XRPL Payment and then crashed before it wrote the hash and the credit, a retry could pay again. The unique settlement row was not enough if the chain had already moved and the database had not.

Ack-before-done. Redis entries were acknowledged when the worker *took* the message, not when the database reached `completed` or `failed`. A crash after ack meant the stream was done and the remittance was not. There was no PEL reclaim, so the message sat nowhere useful. Admin retry was failed-only.

Cash-out lied about the ledger. Spendable balance was treated as if it were the on-chain balance. Simulated cash-out never burns tokens, so after a cash-out request the wallet JSON could not tell "I can still spend this" from "this is still sitting on Testnet". Clients had no honest pair of numbers.

`create_all` schema drift. SQLAlchemy `create_all` creates missing tables and then stops. New columns on existing tables never appeared. That already bit development once. Demo databases and the models silently diverged.

Silent decrypt. If a Fernet key was wrong, encrypted columns could come back as empty instead of failing. A bad key looks like "this user has no ID number" rather than "stop, you cannot read the vault".

Other sketch behaviour we also replaced: plaintext session tokens in the sessions table; no tracking reference; cash-in possible on a stale quote; retry that could not unstick pending or processing rows; TrustSet that could generate a new wallet if the first attempt died after address creation.

Table 1 is the short version we used when deciding patch order.

Table 1: Sketch failure versus what this fork does.

Sketch	**This fork**
Quote counted forever; no cancel/TTL	15 min TTL; cancel; expired/cancelled excluded from limits
Crash after Payment could pay twice	Hash short-circuit; memo = remittance id; one credit commit
Ack on take; no PEL reclaim	Ack on completed/failed; reclaim then new entries
Wallet JSON = spendable only, called "ledger"	Spendable vs on-chain (requested/approved reservations)
`create_all` never alters columns	Alembic 0001 then 0002
Bad Fernet key → empty field	`RuntimeError` on decrypt

7 Why Patch A then Patch B

We split the work on purpose.

Patch A was money movement and schema truth: Alembic `0001_initial` (including `stream_entry_id`), outbox republish, Payment memo with remittance id, no second pay if

the hash is set, ack only on terminal status, PEL reclaim with a Redis 5 fallback, retry of pending/processing/failed, persist address before TrustSet, decrypt errors that raise. Those bugs lose value or strand a Payment. They had to be fixed before we added more columns.

Patch B was not allowed to rewrite `0001_initial`. It added `0002_quote_session`: quote TTL, `cancelled`, tracking refs, hashed session tokens, CORS, wallet JSON that distinguishes spendable and on-chain, limit rules that ignore cancelled and expired quotes, 422 when the fee consumes the send. That patch is what senders and the future UI actually feel, but it sits on a ledger that already tells the truth.

Doing B first would have meant either editing 0001 (forbidden once A shipped) or stacking quote columns onto a settlement path that could still double-pay. Doing A first meant the worker invariants were in place, then B could add `expires_at` and `token_hash` without touching Payment code.

The same order is how you should talk about the demo. Show that a Payment is idempotent (retry of completed is 409; a second worker pass does not double-credit) *before* you show the 15-minute quote timer. Markers who only see Swagger quotes will miss why A existed.

8 How to run and demo on Windows

You need Python 3.11+, a local Redis on port 6379, and two Fernet keys. This team's Redis was Memurai / Redis 5.0.14; Docker `redis` or WSL `redis-server` also work. Homebrew is not the Windows path.

1. Start Redis so `redis://localhost:6379/0` answers. Confirm with any Redis CLI `PING`.
2. In PowerShell:

   ```
   cd backend
   python -m venv .venv
   .\.venv\Scripts\Activate.ps1
   pip install -r requirements.txt
   Copy-Item .env.example .env
   ```

3. Generate two different keys and paste them into `.env`. In Python: `Fernet.generate_key()` from the `cryptography` package. Set `KYC_ENCRYPTION_KEY` and `XRPL_KEY_ENCRYPTION_KEY`. Do not commit `.env`.
4. Prefer an empty database file. From `backend`:

   ```
   .\.venv\Scripts\alembic upgrade head
   ```

   App startup runs the same upgrade. Leftover `create_all` files without `alembic_version` will fail; delete them or stamp only if you know why.
5. Terminal 1: start uvicorn with reload from `backend`, then open http://127.0.0.1:8000/docs.
6. Create an admin (never via register): `python -m scripts.create_admin` with name, email, mobile, password.
7. Once per environment, with network access to Testnet:

   ```
   python -m scripts.setup_platform_wallet
   ```

   It prints the classic address only, never the secret. Send that address to Marc for a 100,000 UCTUSD grant (distributor `rsWPX7FKwnfk6enosumAzEuTs5Y12Steq4`). Do not mint a new token.
8. Demo path in Swagger: register two customers, submit KYC, admin-approve both, add a beneficiary that matches the recipient, create a quote, cash-in, admin confirm-cash-in, then either

   ```
   python -m scripts.run_settlement_worker
   ```

   or `POST /admin/settlement/run`. Recipient `GET /wallet/me` should show the hash and spendable balance. Completing a cash-out burns reserved UCTUSD on-chain (Payment to the issuer).
9. Tests (Redis must be up; XRPL is mocked):

   ```
   python -m pytest -q
   ```

   Expect 129 passed.

If settlement is stuck in pending or processing, retry that settlement id as admin. Do not retry completed. If Redis was down during enqueue, retry republishes the outbox row.

8.1 Ten-minute marking demo

With Redis up, keys set, admin created, and the platform wallet funded: (1) register sender and recipient; (2) both submit KYC; (3) admin approves both; (4) sender adds the recipient as a beneficiary (mobile/email match); (5) post a quote under R3,000; (6) show that cancel frees the limit, then create a fresh quote; (7) cash-in, admin confirm; (8) run the worker; (9) recipient wallet shows spendable, hash, and on-chain; (10) request cash-out, admin complete—spendable already dropped at request; complete burns on-chain so that balance drops too; (11) fail a second cash-out and show the refund. Optional: cancel is 409 after expiry; retry of completed is 409.

The pytest suite is the regression net for those stories (auth, KYC, quotes, limits, cash-in, settlement, wallet, cash-out, CORS, crypto). XRPL is mocked; Redis is not. Expect 129 passed.

Compiled reports: `docs/reports/spec/` and `docs/reports/rationale/` PDFs. `PERFORMANCE_TESTING.md` has the 7 September 2026 re-run on this fork (HTTP load, enqueue, and five live Testnet settlements). Kerry's 31 August / 1 September tables stay in the same file as the sketch baseline.

References

FinMark Trust (2024). South africa to the rest of SADC remittances market assessment 2024. Technical report, FinMark Trust.

International Monetary Fund (2025). South africa: Technical assistance report—cross-border payments: South africa–zimbabwe corridor diagnostic. Technical Report 25/93, International Monetary Fund.

Rateweb (2026). Your transfer fee ignores where you're sending. that costs you. Online article analysing World Bank Remittance Prices Worldwide 2025 Q1 figures for South African outbound corridors. Secondary citation of World Bank RPW 2025 Q1. Reports Western Union and MoneyGram flat fees of about R214–R217 on a surveyed R1,370 send.

University of Cape Town (2026a). ECO5040W class project brief: XRPL-based FX remittance platform using RLUSD. Course brief, Financial Software Engineering.

University of Cape Town (2026b). ECO5040W project resources. Trust lines, IOUs, RLUSD vs fallback IOU, recommended stack.

World Bank (2025). Remittance prices worldwide. Technical report, World Bank. Quarterly survey of the cost of sending remittances. Primary source for corridor fee comparisons.

XRPL Foundation (2026a). Issued currencies. XRP Ledger documentation.

XRPL Foundation (2026b). Trust lines. XRP Ledger documentation.